

Angular Moderno

26 - Validações

2026

SENAI - CRTI

Aula 26 – Validações

Nesta aula vamos evoluir o formulário de checkout adicionando validações para garantir a qualidade dos dados e melhorar a experiência do usuário.

Validações Obrigatórias

Aplicar campos required no formulário

Regras de Validação

Criar regras específicas por campo

Validação Customizada

Implementar lógica personalizada

UX Melhorada

Feedback visual ao usuário



Problema atual: O usuário consegue enviar o formulário sem preencher corretamente os dados. Vamos resolver isso agora!



Importância das Validações

Validações são essenciais para garantir qualidade dos dados e melhorar a experiência do usuário.

- ✓ Obrigatórias (required)
Garantem que campos essenciais não sejam deixados em branco pelo usuário.
- @ Formato (email)
Verificam se o valor inserido segue o padrão esperado, como um endereço de e-mail válido.
- 📏 Tamanho (minLength)
Definem um número mínimo de caracteres para garantir dados suficientes.
- ⚙️ Customizadas
Regras específicas do negócio que não são cobertas pelas validações padrão do Angular.

📌 Neste momento estamos validando apenas no **frontend** para melhorar a experiência do usuário. Em aplicações reais, as validações também devem ser feitas no **backend**.



Etapa 0: Parar o Servidor

Antes de iniciar as alterações, pare o servidor Angular para evitar erros durante as modificações no formulário.

Comando no Terminal

CTRL + C

Execute este atalho no terminal onde o servidor Angular está rodando.

Por que parar o servidor?

- Evitar erros durante alterações no formulário
- Garantir que o Angular reconheça corretamente as mudanças
- Prevenir conflitos de compilação durante edições

Etapa 1: Importar Validators

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

Atualize os imports do arquivo para incluir `Validators`, `AbstractControl` e `ValidationErrors`.

```
import { Component, inject } from '@angular/core';
import {
  ReactiveFormsModule,
  FormGroup,
  FormControl,
  Validators,
  AbstractControl,
  ValidationErrors,
} from '@angular/forms';

import { CarrinhoService } from '../../../core/services/carrinho.service';
```

Validators

Fornece validações prontas do Angular como `required`, `email` e `minLength`.

AbstractControl

Usado como tipo de parâmetro para funções de validação customizada.

ValidationErrors

Define o tipo de retorno dos erros em validações customizadas.

Etapa 2: Criar Validação Customizada

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

Criar função para impedir números no campo nome. Esta função será usada como validador customizado no formulário.

```
function nomeSemNumeros(control: AbstractControl): ValidationErrors | null {  
  
  const valor = control.value;  
  
  if (!valor) return null;  
  
  if (/\\d/.test(valor)) {  
    return { numeroInvalido: true };  
  }  
  
  return null;  
}
```

1

Recebe o controle

`AbstractControl` representa qualquer campo do formulário

2

Testa com Regex

A expressão `/\\d/` detecta a presença de números no valor

3

Retorna resultado

Erro `numeroInvalido: true` se inválido, ou `null` se válido

Etapa 3: Aplicar Validações no Formulário

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

Atualizar o `FormGroup` com as validações em cada campo do formulário de checkout.

```
formulario = new FormGroup({
  nome: new FormControl('', [Validators.required, Validators.minLength(3), nomeSemNumeros]),
  email: new FormControl('', [Validators.required, Validators.email]),
  endereco: new FormControl('', [Validators.required, Validators.minLength(5)]),
});
```

required Torna o campo obrigatório — não pode ser enviado vazio	minLength Define tamanho mínimo de caracteres para o campo
email Valida o formato correto de endereço de e-mail	nomeSemNumeros Validação customizada que impede números no nome

Etapa 4: Bloquear Envio Inválido

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

Atualizar o método `finalizar()` para impedir o envio do formulário quando os dados estiverem inválidos.

```
finalizar() {  
  if (this.formulario.invalid) {  
    console.log('Formulário inválido');  
    return;  
  }  
  
  const dados = this.formulario.value;  
  const itens = this.carrinhoService.itens();  
  
  console.log('Dados do formulário:', dados);  
  console.log('Itens do carrinho:', itens);  
}
```

`formulario.invalid`

Verifica todas as validações de todos os campos do formulário de uma só vez. Se qualquer campo for inválido, retorna `true`.

Benefícios

- Impede execução com dados inválidos
- Melhora o controle do fluxo de compra
- Evita chamadas desnecessárias ao backend

Etapa 5: Atualizar Template – Nome

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

Adicionar as mensagens de validação para o campo nome no template HTML.

```
<p>
  <input type="text" formControlName="nome" placeholder="Nome" />
</p>

@if (formulario.get('nome')?.touched && formulario.get('nome')?.invalid) {
  @if (formulario.get('nome')?.errors?.['required']) {
    <small>Nome é obrigatório</small>
  }

  @if (formulario.get('nome')?.errors?.['minlength']) {
    <small>Nome deve ter no mínimo 3 caracteres</small>
  }

  @if (formulario.get('nome')?.errors?.['numeroInvalido']) {
    <small>Nome não pode conter números</small>
  }
}
```

touched

Garante que o usuário interagiui com o campo antes de exibir erros, evitando mensagens prematuras.

errors

Permite acessar cada tipo de erro individualmente para exibir mensagens específicas e contextuais.

Mensagens controladas

Cada erro exibe uma mensagem diferente, orientando o usuário sobre o que precisa ser corrigido.

Etapa 6: Atualizar Template – Email

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

Adicionar as mensagens de validação para o campo e-mail no template HTML.

```
<p>
  <input type="email" formControlName="email" placeholder="E-mail" />
</p>

@if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
  @if (formulario.get('email')?.errors?.['required']) {
    <small>Email é obrigatório</small>
  }

  @if (formulario.get('email')?.errors?.['email']) {
    <small>Email inválido</small>
  }
}
```

Validador required

Valida se o campo está vazio. Exibe a mensagem *"Email é obrigatório"* quando o usuário não preenche o campo.

Validador email

Valida o formato do e-mail. Exibe a mensagem *"Email inválido"* quando o formato não corresponde ao padrão esperado.

Etapa 7: Atualizar Template — Endereço

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

Adicionar as mensagens de validação para o campo endereço no template HTML.

```
<p>
  <input type="text" formControlName="endereco" placeholder="Endereço" />
</p>

@if (formulario.get('endereco')?.touched && formulario.get('endereco')?.invalid) {
  @if (formulario.get('endereco')?.errors?.['required']) {
    <small>Endereço é obrigatório</small>
  }

  @if (formulario.get('endereco')?.errors?.['minlength']) {
    <small>Endereço deve ter no mínimo 5 caracteres</small>
  }
}
```

Campo Endereço

Obrigatório e com mínimo de 5 caracteres para garantir um endereço minimamente descritivo.

Erro required

Exibe "Endereço é obrigatório" quando o campo está vazio após interação.

Erro minlength

Exibe "Endereço deve ter no mínimo 5 caracteres" quando o texto é muito curto.

Etapa 8: Desabilitar Botão

SRC/APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

Adicionar o binding `[disabled]` ao botão de finalizar compra para impedir o envio quando o formulário estiver inválido.

```
<p>
  <button type="button" (click)="finalizar()" [disabled]="formulario.invalid">
    Finalizar Compra
  </button>
</p>
```

`[disabled]="formulario.invalid"`

O binding de propriedade `[disabled]` desabilita o botão automaticamente sempre que qualquer campo do formulário estiver inválido. Quando todos os campos forem válidos, o botão é habilitado automaticamente.

Benefícios para o usuário

- Feedback visual imediato sobre o estado do formulário
- Impede cliques acidentais com dados incompletos
- Melhora significativamente a experiência de compra



Etapa 9: Executar o Projeto

Com todas as alterações salvas, é hora de iniciar o servidor e testar o formulário com validações no navegador.

Comando no Terminal

```
ng serve
```

Execute este comando na raiz do projeto Angular para iniciar o servidor de desenvolvimento.

Acessar no Navegador

```
http://localhost:4200/checkout
```

Abra o endereço acima no navegador para visualizar o formulário de checkout com as validações aplicadas.

Testes do Formulário

Após executar o projeto, realize os seguintes testes para garantir que todas as validações estão funcionando corretamente.

1

Campo vazio

Clique em um campo e saia sem preencher. Deve exibir mensagem de erro obrigatório.

2

Nome curto

Digite menos de 3 caracteres no nome. Deve exibir erro de tamanho mínimo.

3

Nome com números

Digite números no campo nome. Deve exibir erro da validação customizada.

4

Email inválido

Digite um e-mail sem o formato correto. Deve exibir mensagem de email inválido.

5

Botão desabilitado

Verifique que o botão está desabilitado enquanto o formulário estiver inválido.

6

Envio válido

Preencha todos os campos corretamente. O botão deve habilitar e o envio deve funcionar.

Resultado da Aula

Após esta aula, o sistema de checkout agora possui um conjunto completo de validações que garantem a qualidade dos dados e melhoram a experiência do usuário.



Campos Obrigatórios

Validação de campos obrigatórios com `required`



Validação de Formato

Verificação de formato correto para e-mail



Validação de Tamanho

Controle de tamanho mínimo com `minLength`



Validação Customizada

Regra específica para impedir números no nome



Interface Reativa

Feedback visual imediato ao usuário durante o preenchimento

Dúvidas

?

